



BTGenBot × NAV4RAIL

Semaine 1 — EDA et préparation du dataset

Benjamin Lepourtois Anne Faury Mohamed Amar
Mourad Latoundji

28 mai 2026

Mastère spécialisé IA/DATA — Promotion 2026



Plan de travail et posture

Dataset et chiffres clés

Validité XML et structure

Catégorisation et recovery patterns

Token count

Synthèse

Objectifs annoncés :

- Téléchargement dataset BTGenBot
- EDA : catégories, longueur BTs, vocabulaire nœuds
- Conversion JSONL + adaptation prompt

Livrable concret :

- 1 notebook synthèse unifié
- Dataset SFT-ready
- Rapports MD + figures réutilisables

Posture méthodologique

Fidélité maximale au papier
BTGenBot (Izzo 2024).

Le papier ne mentionne aucun filtrage, le repo officiel ne livre pas le script d'entraînement.

On garde les **594 exemples bruts**, on documente tout le reste.

Plan de travail et posture

Dataset et chiffres clés

Validité XML et structure

Catégorisation et recovery patterns

Token count

Synthèse

Source :

- Dépôt officiel
AIRLab-POLIMI/BTGenBot
- `dataset/bt_dataset.json`, 2.2 MB
- Diff byte-à-byte vs fichier local → identique

Format : Alpaca

- `instruction` (consigne système)
- `input` (description NL de la tâche)
- `output` (Behavior Tree en XML)

Volume : 594 paires (papier annonce
~600)

Origine des BTs

Behavior Trees extraits de projets ROS2 open-source (Nav2, ROBORTS, etc.).

Descriptions NL générées par GPT-3.5-turbo à partir des BTs.

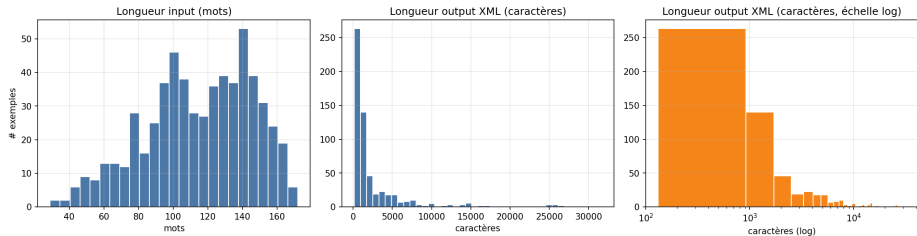
⇒ Dataset **semi-synthétique**.

Métrique	Valeur
Total exemples	594
XML valides (parse strict)	578 (97.31%)
XML valides (cleaning étendu)	571 (96.13%)
Longueur input (mots), médiane	117
Longueur input (mots), p95	158
Longueur XML (chars), médiane	1 080
Longueur XML (chars), p95	11 878
Longueur XML (chars), max	31 681
Nb nœuds BT, médiane / max	15 / 399
Profondeur BT, médiane / max	5 / 21
Tags XML uniques	360
Skills métier uniques	1 223
Variables blackboard {var} uniques	296

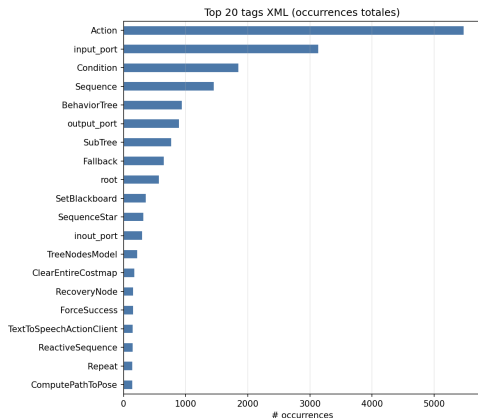
Lecture

- Inputs **très homogènes** (médiane 117 mots, p95 158)
- XML **très asymétrique** : médiane 1k chars, max 31k
- Arbres variés : 15 nœuds typique, jusqu'à 399 (Nav2 cascadié)
- **1 223 skills uniques** = vocabulaire ouvert (vs catalogue fermé SNCF, 28 skills)

Longueurs : input vs output XML



Input : distribution **gaussienne resserrée** (prompt court généré uniformément par GPT-3.5). Output : distribution **log-normale** avec longue traîne — quelques BTs Nav2 monstrueux dominant les statistiques.



Couche	Volume
Tags XML uniques (total)	360
Nœuds de contrôle BT.CPP	~12
Skills domaine uniques	1 223
Variables blackboard {var}	296

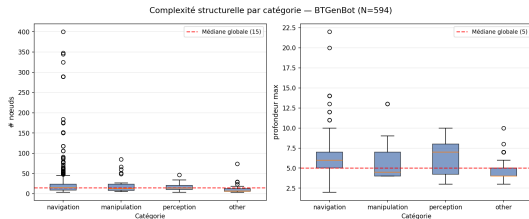
Occurrences dans le dataset :

Action	2 605
Sequence	855
SubTree	622
Fallback	299

Deux niveaux d'abstraction

Syntaxe structurée (12 nœuds, stable)
+ vocabulaire ouvert (1 223 skills, domaine-dépendant). Le modèle doit apprendre les deux simultanément.

Complexité structurelle par catégorie



Catégorie	Nœuds	Prof.
navigation (N=434)	14	6
+ recovery (192)	22	7
manipulation (N=43)	12	5
perception (N=15)	13	7
other (N=100)	6	4
NAV4RAIL réel	112	~7

Perception : profondeur =
navigation+recovery avec **15× moins**
d'exemples — variance élevée.

NAV4RAIL réel hors-échelle : 7× plus grand
que la médiane dataset.

Plan de travail et posture

Dataset et chiffres clés

Validité XML et structure

Catégorisation et recovery patterns

Token count

Synthèse

Distribution des erreurs :

Type d'erreur	Count
Token invalide (commentaires <code><!--></code> malformés, attributs sans guillemets)	12
Tag non fermé / mal fermé	4
Total invalides	16 (2.7%)

Origine : bugs de génération
GPT-3.5 — XML « presque valide ».
Cohérent avec dataset
semi-synthétique.

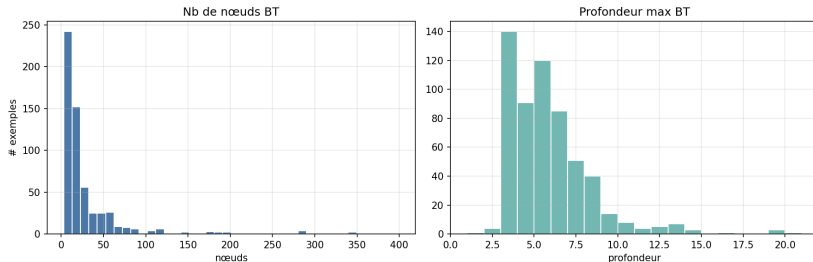
Décision

Dataset conservé à 594 entiers.

Le papier ne filtre pas. SFTTrainer apprend du texte, ne valide pas le XML. Les 16 invalides (2.7%) seront appris comme texte ordinaire — bruit négligeable.

On documente, on filtre pas.

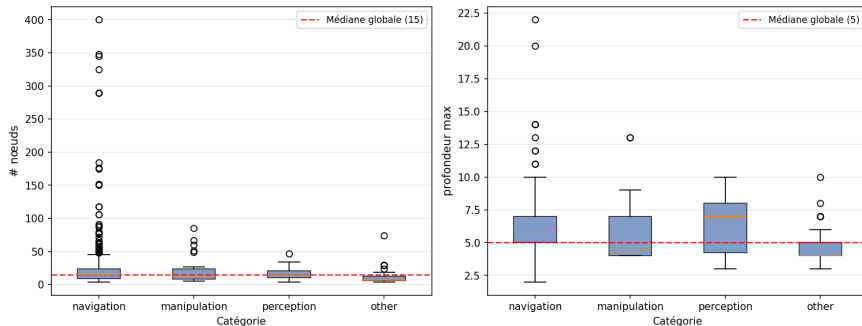
Structure des Behavior Trees



La plupart des BTs sont **petits à moyens** (médiane 15 nœuds, profondeur 5) mais quelques arbres atteignent 399 nœuds et 21 niveaux — scénarios Nav2 complets avec recovery cascadé.

Métriques de complexité — distribution par catégorie

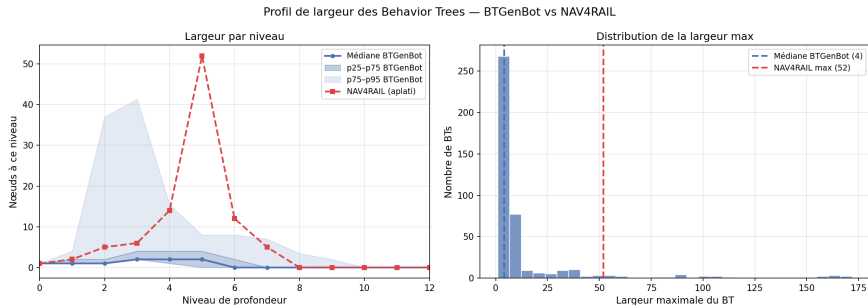
Complexité structurelle par catégorie — BTGenBot (N=594)



Navigation avec recovery : médiane **22 nœuds**, profondeur **7** — 60% plus complexe que navigation simple. Perception : profondeur élevée (7) sur seulement **15 exemples** — variance forte, signal faible.

Ligne rouge = médiane globale du dataset. Le BT NAV4RAIL réel (112 nœuds) est hors-échelle.

Largeur des BTs — profil par niveau de profondeur



BTGenBot : largeur médiane max = 4 nœuds (p95 = 55). Distribution très resserrée : la majorité des BTs restent étroits. **Exemple NAV4RAIL** : pic à profondeur 5 avec **52 nœuds** (9 primitives de mouvement $\times$ actions imbriquées) — **13 $\times$ plus large** que la médiane dataset.

⇒ Le modèle entraîné ne génère jamais de BTs aussi larges : risque de génération de structures linéaires (une branche unique) pour des tâches multi-primitives.

Dans BTGenBot (594 BTs) :

Tag	Occurrences
SubTree (API v3)	622 (dans 90 BTs, 15%)
SubTreePlus (API v4)	0

Dans NAV4RAIL réel :

Tag	Occurrences
SubTreePlus	15
Architecture entièrement modulaire	
__autoremap="true" + remapping ports	

Conséquence pour la génération

Le modèle ne produira **jamais** SubTreePlus spontanément.

Il générera soit :

- un arbre **monolithique** plat (distribution entraînement)
- du SubTree v3 (présent à 15% en training)

⇒ Sans contrainte explicite dans le prompt, les BTs générés sont **structurellement incompatibles** avec l'architecture NAV4RAIL.

Plan de travail et posture

Dataset et chiffres clés

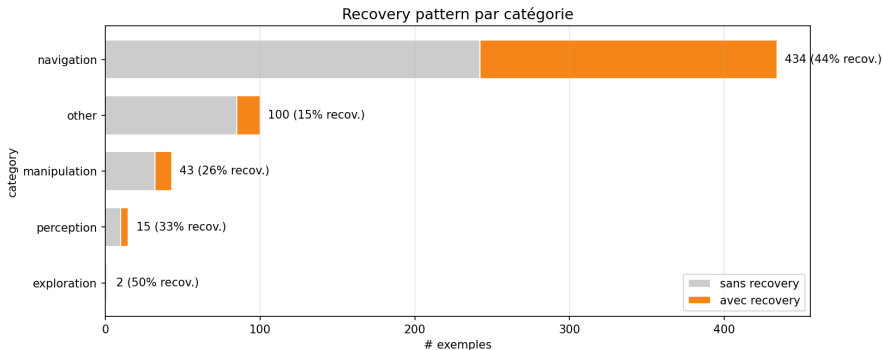
Validité XML et structure

Catégorisation et recovery patterns

Token count

Synthèse

Catégories descriptives — méthode hybride



Méthode : signal XML (skills présents) prioritaire, fallback regex sur input texte. 5 catégories canoniques.

Flag transverse `has_recovery_pattern` = présence de `Fallback/RecoveryNode` avec `Sequence` imbriquée : **224 / 594** BTs (37.7%), concentrés sur navigation (44% des BTs nav).

Distribution :

Catégorie	%
navigation	73.1%
other	16.8%
manipulation	7.2%
perception	2.5%
exploration	0.3%

NAV4RAIL = navigation + perception

SNCF = inspection ferroviaire.

Dataset BTGenBot **bien aligné navigation** (73%) mais **faible perception** (15 exemples, 2.5%).

⇒ Mauvaise nouvelle pour le transfert direct sur les skills perception du catalogue SNCF.

Le modèle apprendra solidement les patterns Nav2 mais aura peu de signal pour la capture / analyse visuelle propres à l'inspection.

Anatomie comparative — BTGenBot vs NAV4RAIL réel

	BTGenBot (médiane dataset)	NAV4RAIL (behavior_tree_example.xml)	réel
Définitions BT	1	16 (main + 15 sous-arbres)	
Nœuds totaux	15	112	
Profondeur max	5	~7 par sous-arbre	
Largeur max	4 (médiane)	52 (exemple NAV4RAIL, profondeur 5)	
SubTreePlus	0	15	
Vocabulaire actions	Nav2 (ClearEntireCostmap...)	Inspection (ManageMeasurements...)	
Logique inspection	absente	centrale	

⇒ NAV4RAIL **7× plus grand** (nœuds), largeur **13× plus large** au pic (52 vs 4), entièrement modulaire via SubTreePlus, vocabulaire métier totalement distinct.

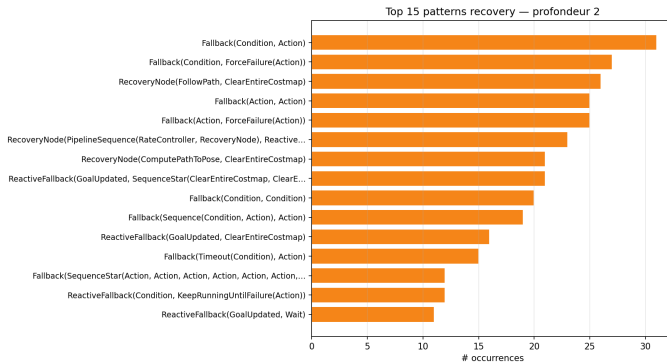
Bilan transférabilité — ce qui passe, ce qui ne passe pas

	Éléments	Justification
✓ Transférable	Nœuds de contrôle : Sequence, Fallback, ReactiveFallback, Repeat	Identiques BT.CPP 3.x et 4.x — syntaxe partagée
~ Partiel	Structure recovery Fallback(Seq, Action)	Motif présent (38% des BTs) mais actions Nav2-spécifiques
× Non transférable	Vocabulaire action/condition (0/43 skills NAV4RAIL dans training) ; SubTreePlus ; logique inspection	Jamais vu en training

Quelles pistes d'adaptation ?

Le modèle doit être guidé sur le vocabulaire et la structure NAV4RAIL. Mais comment ? **Prompt engineering** avec la liste des skills SNCF ? **Second SFT** sur quelques exemples NAV4RAIL ? **RAG** sur les spécifications des BTs cibles ? La réponse dépend du volume d'exemples disponibles — aujourd'hui : **1 seul**.

Recovery patterns — Top motifs Nav2



Top : `Fallback(Condition, Action)` (31), `RecoveryNode(FollowPath, ClearEntireCostmap)` (26), `RecoveryNode(ComputePathToPose, ClearEntireCostmap)` (21). $\Rightarrow$ Signature canonique du recovery Nav2. Le modèle apprend la **syntaxe**, pas la sémantique. Ces patterns 2D ne sont pas transférables tels quels à NAV4RAIL : seule l'idée de séquence corrective reste pertinente.

Plan de travail et posture

Dataset et chiffres clés

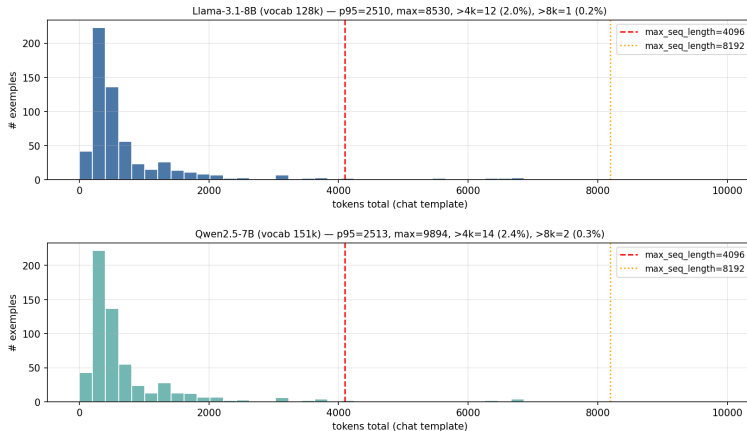
Validité XML et structure

Catégorisation et recovery patterns

Token count

Synthèse

Tokenization — diagnostic critique vs papier



Papier fixe `max_seq_length=4096` sans documenter la distribution. **Notre mesure** : Llama-3.1 et Qwen2.5 quasi-équivalents (ratio médian = 1.000). Troncature à 4096 : 2.0–2.4%. $\Rightarrow$ On garde 4096 (fidélité), on documente le taux. Ablation 8192 si VRAM le permet en QLoRA.

Plan de travail et posture

Dataset et chiffres clés

Validité XML et structure

Catégorisation et recovery patterns

Token count

Synthèse

Livrables produits :

- 1 notebook synthèse unifié
(`eda_BT_synthesis_Benjamin.ipynb`)
- 2 JSONL trainables (chat + Alpaca)
- 8 figures PNG (`dataset/figs/`)
- 5 tables CSV + 7 rapports MD
- 1 smoke test set (15 BTs)
- 1 fichier `prompts.json` (v1 + v2)

Apports vs papier :

- Distribution token explicite (papier muet)
- Catégorisation hybride XML+texte
- Recovery patterns à profondeur 3
- Diagnostic transfert SNCF chiffré

Prochaine étape — semaine 2

Fine-tuning SFT QLoRA sur cluster
Télécom Paris (RTX 3090).

3 modèles :

- Llama-3.1-8B-Instruct
- Qwen2.5-7B-Instruct
- Qwen2.5-Coder-7B-Instruct

Config papier : LoRA $r=8/\alpha=16$, 70–80
epochs, `max_seq` 4096.

⇒ Dataset prêt, code de loading direct
via `datasets.load_dataset`.